Insertion Sort — Implementation & Analysis Report

Generated: 2026-03-04T00:16:01.672Z

Part 1 — Implementation Summary

Two Java implementations were built and called from JavaScript (Node.js):

- **Regular Insertion Sort** (stable by shifting only elements > key)
- **Binary Insertion Sort** (optimized comparisons using upper-bound binary search; still stable)

Small array sanity check:

- Regular sortedOk: `true`, result: `[1,2,5,5,6,9]`
- Binary sortedOk: `true`, result: `[1,2,5,5,6,9]`

Part 2 — Performance Analysis (Metrics from Java)

Metrics tracked: elapsed time (ns), comparisons, writes/moves.

Best-case (Nearly Sorted)

Insertion sort performs well here because very few elements need shifting; the inner loop exits quickly.

Worst-case (Reversed)

This forces maximum shifting: each new key moves to the front, producing ~$O(n^2)$ writes and comparisons.

Summary table (best vs worst)

Scenario	n	Regular ms	Regular comparisons	Regular writes	Binary ms	Binary comparisons
Best (nearly)	5000	3.520	133530	133530	4.357	54354
Worst (reversed)	5000	32.646	12497500	12502499	16.121	56809

Summary table (best vs worst) Continues...

Scenario	Binary writes
Best (nearly)	133530
Worst (reversed)	12502499

Average-case (Random arrays)

Average-case remains **$O(n^2)$**, but the binary variant typically reduces comparisons (still must shift).

n	Regular ms	Regular comparisons	Regular writes	Binary ms	Binary comparisons	Binary writes
200	0.445	10182	10189	0.478	1254	10117
500	2.938	63228	63235	2.256	3790	63531
1000	5.441	251582	251590	4.007	8573	252776
2000	8.106	1008869	1008879	6.087	19157	999441

Space Complexity Discussion

Both implementations are **in-place**: they sort within the array using a constant amount of extra memory (O(1) auxiliary space).

Stability Analysis

A Java stability demo sorts pairs by key and returns the relative order of tags with the same key.

- Expected order for key=2: `["A","B","C"]`
- Observed: `["A","B","C"]`

Part 3 — Reflection

Efficiency Discussion

- **Small arrays:** insertion sort is often very competitive due to low overhead and cache-friendly behavior.
- **Large arrays:** $O(n^2)$ growth becomes expensive; algorithms like QuickSort/MergeSort are typically preferred.
- **Binary insertion:** reduces comparisons but does not eliminate $O(n^2)$ shifts, so improvements are noticeable mostly in comparison-heavy contexts.

Practical Applications

- Sorting small lists (UI items, short buffers).
- Nearly-sorted datasets (incremental updates).
- As a building block in hybrid sorts (e.g., TimSort uses insertion sort on small runs).

Improvements and Variations

- **Binary insertion sort:** reduces comparisons using binary search (implemented here).
- **Hybrid algorithms:** TimSort combines merge strategies with insertion sort for small partitions/runs.
- **Sentinel optimization / gap insertion variants:** reduce bounds checks or shift patterns in low-level implementations.

Code Snippets (from Java)

Regular stable inner loop

```
while (j >= 0) {
    comparisons++;
    if (a[j] > key) {
        a[j + 1] = a[j];
        writes++;
        j--;
    } else {
        break;
    }
}
```

Binary insertion stable position (upper bound)

```
if (a[mid] <= key) {
    lo = mid + 1;
} else {
```

```
    hi = mid;
}
```

How to Reproduce

GitHub Link:

```
https://github.com/DiscipleA/IT212/blob/main/Week9/insertion-sort-
assignment/README.md
```

1) Build Java

```
cd java
mvn -q -DskipTests package
```

2) Install JS dependencies

```
cd ../js
npm install
```

3) Run tests

```
npm test
```

4) Generate the report

```
npm run report
```

The report will be generated at:

```
reports/REPORT.md
```